Pivoting Data in Spark

In this train, we'll learn how to pivot data for analysis using Spark. Pivoting allows you to transform rows into columns, typically to summarize or reshape datasets for downstream reporting and analysis.

Learning objectives

By the end of this train you should be able to:

- [Placeholder for objective 1]
- [Placeholder for objective 2]
- [Placeholder for objective 3]

Learning Objectives

- By the end of this train, you should be able to pivot data using Spark.
- Understand the differences between narrow (long) and wide data formats.
- Use `groupBy().pivot().agg()` to reshape and summarize data efficiently.

Understanding Data Pivoting in Spark

Pivoting in Spark involves transforming a DataFrame from a long format (rows for each category) to a wide format (each category becomes a column). This is often used for reporting or preparing data for ML models and dashboards.

Use case: Suppose we have temperature readings per hour for each county. Pivoting allows us to create a summary like mean temperature per county in a columnar format.

The inverse of pivoting is called unpivoting (or melting), which reshapes data from wide back to long format.

Pivoting Data: Example in Spark

```
from pyspark.sql import functions as F

# Sample DataFrame of temperature readings
df = spark.createDataFrame([
    ("Dublin", "2024-01-01", 8),
    ("Dublin", "2024-01-02", 7),
    ("Cork", "2024-01-01", 10),
    ("Cork", "2024-01-02", 9)
], ["county", "date", "temp"])

# Pivot the data to show average temperature per date for each county
pivoted = df.groupBy("date").pivot("county").agg(F.avg("temp"))
pivoted.show()
```

Explanation: - `groupBy("date")` groups the records by date. - `pivot("county")` turns the unique values in the `county` column into new columns. - `agg(F.avg("temp"))` computes the average temperature for each county on each date.

Output:

date	Cork	Dublin
2024-01-01	10	8
2024-01-02	9	7

This format is easier to use for trend analysis or heatmaps in BI tools.

Conclusion

Pivoting data in Spark is a valuable transformation tool that enables analysts and engineers to summarize and reshape datasets efficiently. Whether for reports, dashboards, or preparing data for machine learning models, pivoting helps make insights more accessible.

▼Key Takeaways

- **Takeaway 1:** Pivoting changes the structure of your dataset by turning row values into columns.
- **Takeaway 2:** Use `groupBy().pivot().agg()` to perform summarisation during pivoting.
- **Takeaway 3:** Ideal for visualization or reporting where wide-format data is easier to interpret.
- **Takeaway 4:** Aggregations like mean, count, or sum are commonly applied during pivot operations.
- **Takeaway 5:** The reverse of pivoting is unpivoting, useful when ingesting wide-format data.